

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Experiments

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Total Marks:100

QUESTIONS:4

Annexure A

a) Online Food Delivery System Case study A startup wants to develop an online food delivery platform where users can browse restaurants, place orders, and track delivery in real time. The system should also allow restaurants to update menus and delivery agents to manage orders.

Question: Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

b) Hospital Management System A hospital plans to digitize its operations including patient registration, doctor scheduling, billing, and medical records management. Different users such as doctors, nurses, admin staff, and patients interact with the system.

Question: Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

CASE STUDY (a): ONLINE FOOD DELIVERY SYSTEM

Title

Object-Oriented Analysis and Design of Online Food Delivery System

Aim

To analyze and design an Online Food Delivery System using object-oriented principles and recommend a suitable SDLC model.

Problem Description

A startup wants to develop an online food delivery platform where customers can browse restaurants, place food orders, make payments, and track deliveries in real time. Restaurants should be able to manage menus, and delivery agents should manage deliveries.

Object-Oriented Principles

1. Encapsulation

Encapsulation means hiding data and allowing access through methods.

Example: Order Class

Attributes

- orderID
- orderDate
- totalAmount
- status

Methods

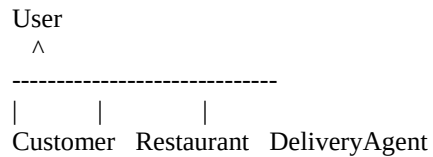
- placeOrder()
- cancelOrder()

- trackOrder()

The order details are protected and can only be accessed through methods.

2. Inheritance

Inheritance allows classes to reuse common properties.



User Class

- userID
- name
- phone

Customer

- browseRestaurant()
- placeOrder()

Restaurant

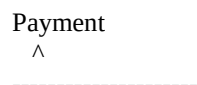
- updateMenu()
- manageOrders()

DeliveryAgent

- acceptOrder()
 - updateDeliveryStatus()
-

3. Polymorphism

The same method performs different actions.



| |
UPIPayment CardPayment

Method

- processPayment()

UPI Payment processes through UPI.

Card Payment processes through card gateway.

This is Runtime Polymorphism.

Key Classes

Customer

Attributes:

- customerID
- name
- address

Methods:

- browseRestaurant()
- placeOrder()

Restaurant

Attributes:

- restaurantID
- name
- location

Methods:

- updateMenu()
- manageOrders()

Menu

Attributes:

- itemID
- itemName
- price

Methods:

- addItem()
- removeItem()

Order

Attributes:

- orderID
- status
- amount

Methods:

- placeOrder()
- cancelOrder()

DeliveryAgent

Attributes:

- agentID
- vehicleNumber

Methods:

- acceptOrder()
- updateStatus()

Payment

Attributes:

- paymentID
- amount

Methods:

- processPayment()

Relationships

Association

Customer ----- Order
Restaurant ----- Menu
DeliveryAgent --- Order

Aggregation

Restaurant <>----- Menu

Inheritance

User
^

| | |
Customer Restaurant DeliveryAgent

Suitable SDLC Model

Agile Model

Justification

- Requirements change frequently.
- New restaurants and offers are added regularly.
- Continuous customer feedback is required.
- Fast development and deployment.
- Easy maintenance and updates.
- Supports iterative releases.

Conclusion

The Online Food Delivery System can be effectively designed using object-oriented concepts such as encapsulation, inheritance, and polymorphism. Classes like Customer, Restaurant, Order, Menu, and DeliveryAgent provide a modular design. Since startup requirements change frequently, the Agile Model is the most suitable SDLC approach.

CASE STUDY (b): HOSPITAL MANAGEMENT SYSTEM

Title

Object-Oriented Analysis and Design of Hospital Management System

Aim

To model a Hospital Management System using object-oriented concepts and recommend an appropriate SDLC model.

Problem Description

A hospital wants to automate patient registration, appointment scheduling, billing, and medical record management. The system will be used by doctors, nurses, patients, and administrators.

Key Classes

Patient

Attributes:

- patientID
- name
- age
- address

Methods:

- register()
 - bookAppointment()
-

Doctor

Attributes:

- doctorID
- specialization

Methods:

- viewAppointments()
 - prescribeMedicine()
-

Nurse

Attributes:

- nurseID
- department

Methods:

- updatePatientRecord()
-

Admin

Attributes:

- adminID
- name

Methods:

- manageUsers()
 - generateReports()
-

Appointment

Attributes:

- appointmentID
- date
- time

Methods:

- scheduleAppointment()
 - cancelAppointment()
-

MedicalRecord

Attributes:

- recordID
- diagnosis
- treatment

Methods:

- addRecord()
 - updateRecord()
-

Bill

Attributes:

- billID
- amount

Methods:

- generateBill()
 - calculateAmount()
-

Relationships

Association

Patient ----- Appointment
Doctor ----- Appointment
Patient ----- MedicalRecord

Aggregation

Hospital <>----- Doctor
Hospital <>----- Nurse
Hospital <>----- Patient

Composition

Patient ♦----- MedicalRecord

Medical records cannot exist without a patient.

Inheritance

User
^

| | | |
Patient Doctor Nurse Admin

Polymorphism

Bill Class

Bill
 calculateBill()

InsuranceBill
 calculateBill()

SelfPaymentBill
 calculateBill()

The method **calculateBill()** behaves differently depending on the payment type.

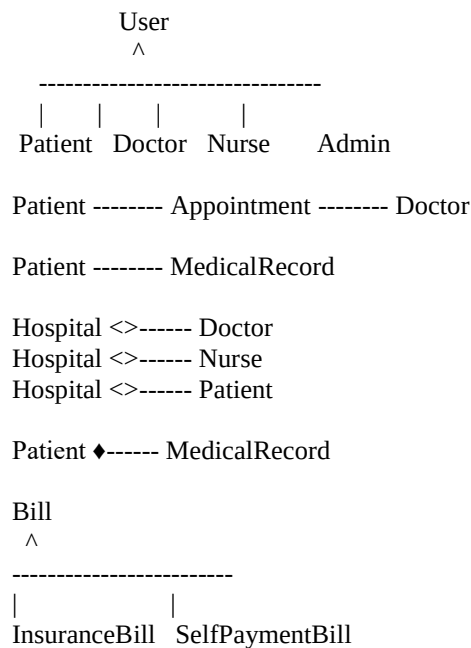
Suggested SDLC Model

Waterfall Model

Justification

- Hospital requirements are generally stable.
- Strong documentation is required.
- Easier compliance with healthcare regulations.
- Well-defined development phases.
- Better control and monitoring.

Class Diagram Structure



Conclusion

The Hospital Management System can be modeled effectively using object-oriented concepts such as inheritance, aggregation, composition, and polymorphism. Key classes include Patient, Doctor, Nurse, Admin, Appointment, MedicalRecord, and Bill. Since hospital requirements are structured and documentation is critical, the Waterfall Model is the most suitable SDLC approach for developing this system.

(a) ONLINE FOOD DELIVERY SYSTEM	
1. Project description	1.1. Project description
2. Project objectives	2.1. Project objectives
3. Project scope	3.1. Project scope
4. Project risks	4.1. Project risks
5. Project budget	5.1. Project budget
6. Project timeline	6.1. Project timeline
7. Project stakeholders	7.1. Project stakeholders
8. Project communication	8.1. Project communication
9. Project monitoring and control	9.1. Project monitoring and control
10. Project closure	10.1. Project closure

CASE STUDY

A startup wants to develop an online food delivery platform where users can browse restaurants, place orders, and track delivery in real time. The system should also allow restaurants to update menus and delivery agents to manage orders.

OBJECT ORIENTED PRINCIPLES

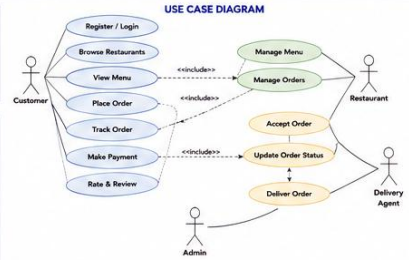
Encapsulation

- Each class hides its data using private attributes and exposes public methods.
- Example: Order class encapsulates order details; only necessary methods like placeOrder(), cancelOrder() are exposed.

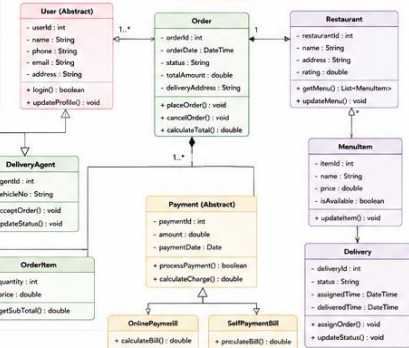
Inheritance

- Common features are placed in a superclass and specialized features in subclasses.
- Example: User is superclass. Customer, Restaurant, DeliveryAgent inherit common properties like id, name, phone, email.

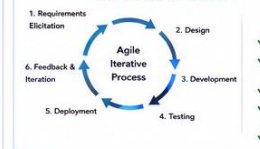
- Same method behaves differently based on the object.
- Example: `calculateCharge()` in `Payment` methods (`OnlinePayment`, `CODPayment`) computes charges differently.



CLASS DIAGRAM



SDLC MODEL: AGILE MODEL



JUSTIFICATION

- ✓ Agile supports iterative releases and continuous improvement.
- ✓ Food delivery platforms require frequent updates (new restaurants, offers, features).
- ✓ Customer feedback can be quickly incorporated.
- ✓ Enables early delivery of working modules like order placement, tracking, payments.
- ✓ Suitable for dynamic and fast-changing requirements.

CONCLUSION

The Online Food Delivery System is effectively modeled using OOP principles. Agile SDLC ensures iterative development, quick adaptation to changes, and continuous customer satisfaction.

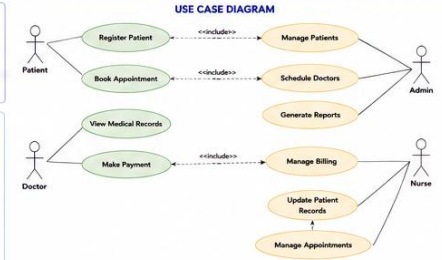
(b) HOSPITAL MANAGEMENT SYSTEM

CASE STUDY

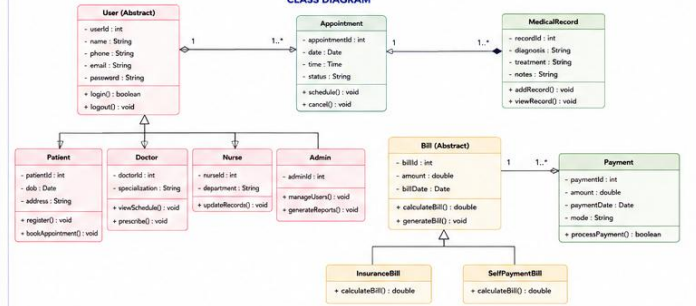
A hospital plans to digitize its operations including patient registration, doctor scheduling, billing, and medical records management. Different users such as doctors, nurses, admin staff, and patients interact with the system.

OBJECT ORIENTED DESIGN

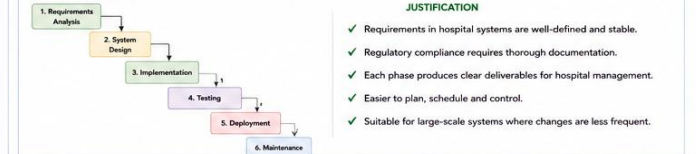
- Encapsulation: Patient data, medical records, and billing details are encapsulated within classes and accessed through methods.
- Inheritance: User superclass with specialized subclasses (Doctor, Nurse, Admin, Patient).
- Polymorphism: Same method like calculateBill() can behave differently in InsuranceBill and SelfPaymentBill.



CLASS DIAGRAM



SDLC MODEL: WATERFALL MODEL



1. Requirements JUSTIFICATION

-
- ```

graph TD
 1[1. Analysis] --> 2[2. System Design]
 2 --> 3[3. Implementation]
 3 --> 4[4. Testing]
 4 --> 5[5. Deployment]
 4 --> 3

```
- ✓ Requirements in hospital systems are well-defined and stable.
  - ✓ Regulatory compliance requires thorough documentation.
  - ✓ Each phase produces clear deliverables for hospital management.
  - ✓ Easier to plan, schedule and control.
  - ✓ Suitable for large-scale systems where changes are less frequent.

## CONCLUSION

The Hospital Management System is modeled using OOP concepts with clear class structures and relationships. Waterfall SDLC ensures disciplined development with proper documentation and reliability.



**Code of Conduct:**

I D. Raghu Vardhan-192311051(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Raghu Vardhan

**RUBRICS FOR CALCULATION:****Case Study**

| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |